

FINAL REPORT – Multi-Level Feedback Queue (MLFQ) Scheduler for xv6-RISC-V

Project Overview

This project involved redesigning xv6-RISC-V's scheduling mechanism by replacing its round-robin policy with a Multi-Level Feedback Queue (MLFQ) scheduler. The updated scheduler introduces multiple priority levels, adaptive promotion/demotion rules, and starvation protection. Additionally, a new system call, `getprocinfo()`, was added to allow user-level programs to query internal scheduling details.

The resulting system provides substantially better performance for I/O-bound and interactive processes while ensuring fairness for CPU-heavy tasks.

1. Design and Implementation

1.1 Modified and Newly Added Components

Several xv6 kernel files were extended or modified:

File	Purpose
<code>kernel/proc.h</code>	Added MLFQ-related fields to both <code>proc</code> and <code>procinfo</code>
<code>kernel/proc.c</code>	Core scheduling logic and MLFQ operations
<code>kernel/param.h</code>	Defined constants for queue count, quantums, and boost interval

`kernel/trap.c` Integrated tick accounting and global boosting mechanism

`kernel/defs.h` Exposed `mlfq_boost()` prototype

`kernel/sysproc` Implemented `getprocinfo()`
`.c`

`kernel/syscall` Added syscall entry
`.*`

`user/user.h` User-level syscall prototype

`user/usys.pl` Syscall stub generation

Newly created files:

- `user/testmlfq.c` – Demonstration and validation tool
- `DESIGN_DOCUMENT` – Main architectural description
- `FINAL_REPORT` – This finalized project summary

1.2 Data Structures Used

Per-process scheduling metadata

```
struct proc {  
  
    int queue_level;           // Priority level (0 = highest)  
  
    int ticks_used;           // Ticks consumed in current time slice  
  
    struct proc *next_proc;    // Linked list pointer for queue  
  
    int quantum;              // Time slice assigned to this queue  
  
};
```

Global queues (4-level MLFQ)

```
struct {  
  
    struct proc *head;  
  
    struct proc *tail;  
  
} mlfq[4];
```

Boost counter

```
static uint boost_counter = 0;
```

1.3 Central Algorithms

Scheduler Loop

The scheduler always selects the runnable process from the highest priority non-empty queue:

```
void scheduler(void) {  
  
    for(;;) {  
  
        p = mlfq_next();      // Retrieve next runnable process (highest priority)  
  
        if(p) {  
  
            p->state = RUNNING;  
  
            swtch(&c->context, &p->context);  
  
            release(&p->lock);  
  
        } else {  
  
            wfi();              // Sleep until next interrupt  
  
        }  
    }  
}
```

Demotion Logic (**yield()**)

A process is demoted only if it uses up its entire quantum:

```

void yield(void) {

    if(p->ticks_used >= p->quantum)

        mlfq_demote(p);

    p->state = RUNNABLE;

    mlfq_enqueue(p, p->queue_level);

    sched();

}

```

Priority Boosting (Timer Interrupt Handler)

After a predefined number of ticks, all processes are lifted back to the highest queue:

During timer interrupts:

- Ticks are incremented
- Global boost threshold is checked
- All processes are moved back to the top queue when needed

```

if(which_dev == 2) { // Timer interrupt

    p->ticks_used++;

    boost_counter++;

    if(boost_counter >= BOOST_INTERVAL) {

        boost_counter = 0;

        mlfq_boost();

    }

    yield();

}

```

2. Evaluation and Testing

2.1 Environment

- **Platform:** xv6-RISC-V
 - **Virtualization:** QEMU
 - **CPUs:** 1 (single core)
 - **Main Test Program:** `testmlfq`
-

2.2 Case Study: CPU-Bound Process

Expected behavior:

- They burn through their quanta quickly
- Repeated demotions push them gradually toward lower queues
- After the boost interval, they return to queue 0

Observed Example Output:

```
CPU bound [PID 3]

queue=0 → ticks=3 of quantum=4

queue=1 → ticks progress 1, 3, 6 of quantum=8

queue=2 → ticks=0 of quantum=16

queue=3 → ticks=32 of quantum=32

... boosted back to queue 0 ...
```

Result: ✓ Correct demotion pattern and recovery after boost.

2.3 Case Study: I/O-Bound Process

These processes frequently sleep, so they rarely consume their entire quantum.

Sample Output:

```
I/O [PID 4]:
```

queue=0, ticks=0/4 (repeated)

Result: ✓ Process reliably stays at the highest queue.

2.4 Case Study: Mixed Workload

Behavioral expectations:

- CPU-heavy process moves downward to lower queues
- I/O process retains high priority
- Both complete without starvation

Result: ✓ Expected scheduling preference observed.

2.5 Testing the System Call `getprocinfo()`

Example usage:

```
struct procinfo info;

getprocinfo(getpid(), &info);

printf("pid=%d queue=%d ticks=%d\n", info.pid, info.queue_level, info.ticks_used);
```

Result: ✓ Syscall accurately reports live scheduler metadata.

3. Key Implementation Challenges

3.1 Safe Lock Ordering

Managing locks during queue operations required strict ordering to avoid deadlocks.

Solution: always acquire `p->lock` only after checking queue status.

3.2 Handling Running Processes During Boost

Running processes are not inside any queue.

Solution: update their `queue_level`, `quantum`, and `ticks_used` without enqueueing.

3.3 Preventing Unfair Penalties for I/O Processes

I/O-bound tasks must not be demoted merely for sleeping.

Solution: demotion only occurs when a process *exhausts* its quantum.

4. Testing Methodology

4.1 Unit-Level Validation

- Queue insertion/deletion
- Quantum enforcement
- Boosting logic correctness
- Syscall memory copying

4.2 Integration Tests

- Single-process runs
- Multi-process competition
- CPU & I/O mixed workloads
- Long execution for starvation detection

4.3 Debugging Tools

- In-kernel `printf` instrumentation
 - The `getprocinfo()` syscall for runtime observability
-

5. Build & Execution Instructions

Compiling

```
make clean
```

```
make qemu
```

Running Tests

Inside the xv6 shell:

```
$ testmlfq
```

Conclusion

The project successfully replaces xv6's default scheduler with a fully working multi-level feedback queue system. The new design demonstrates:

- Improved latency for interactive tasks
- Fair handling of CPU-intensive workloads
- Effective starvation prevention
- Clear instrumentation for debugging (via `getprocinfo`)

This implementation provides a strong foundation for future extensions such as per-queue aging policies, multi-core scheduling improvements, and visualization tools.